

BORG: Diagnosing dependency structure and enforcing leakage-safe cross-validation in R

Gilles Colling ¹

¹ Department of Botany and Biodiversity Research, University of Vienna, Austria

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Cross-validation (CV) estimates how well a model predicts unseen data, but the estimate is only honest when the held-out data are independent of the training data. Ecological, environmental, and clinical data rarely satisfy that assumption: nearby locations, adjacent times, and repeated measurements on the same unit carry shared information. Random k-fold CV then places correlated observations in different folds and reports an error far below what the model achieves in the field ([Roberts et al., 2017](#)).

BORG is an R package that tests for this structure before it splits the data. Given a table and the columns that carry structure (coordinates, a timestamp, a grouping identifier), BORG runs formal dependency tests, classifies the severity, estimates how much random CV would understate the error, and builds a CV scheme that holds the structure out. When a dependency is present, random folds are disabled unless the user overrides explicitly, and the override is logged. BORG also audits full evaluation pipelines for preprocessing, target, and identifier leakage, and writes the diagnosis as a methods paragraph or a machine-readable record for an audit trail.

Statement of need

A defensible spatial or temporal evaluation in R today requires the analyst to already know the data are dependent, choose a blocking scheme, set a block size, and decide to use it. The tooling assumes that decision has been made: `blockCV` ([Valavi et al., 2019](#)), `CAST` ([Meyer et al., 2018](#)), `spatialsample` ([Silge & Mahoney, 2023](#)), `sperrorest` ([Brenning, 2012](#)), and `mlr3spatiotempcv` ([Schratz et al., 2024](#)) generate blocked folds but leave the diagnosis and the choice to the user. Nothing tests the data and refuses an invalid split by default. Leakage through preprocessing or outcome-derived features is a separate and widespread failure mode ([Kapoor & Narayanan, 2023](#); [Kaufman et al., 2012](#)) with no dedicated R tooling at all.

The consequence is a literature of optimistic performance numbers. BORG targets the gap between “a blocking scheme is available” and “the reported number is honest for the intended prediction task.” It makes the diagnosis the default step, quantifies the risk, enforces a matched scheme, and separately guards the preprocessing pipeline. The intended users are ecologists, spatial epidemiologists, and applied machine-learning practitioners who need an evaluation they can defend to a reviewer.

State of the field

The R ecosystem covers fold *generation* well. `blockCV` provides spatial and environmental blocking and a separate routine to estimate the autocorrelation range; `CAST` contributes nearest-neighbour distance matching (`kNNDM`), which adapts folds to the geometry of the prediction

task (Linnenbrink et al., 2024) and the area of applicability (Meyer & Pebesma, 2021);
 spatialsample supplies block and cluster resampling in the tidymodels idiom; sperrorest and
 mlr3spatiotempcv provide spatial partitioning for their respective frameworks. BORG reuses
 this well-developed vocabulary of schemes and adds the layer above it: a diagnosis that decides
 whether and which blocking is warranted, an inflation estimate, an enforcement policy, and a
 pipeline-leakage audit.

Table 1: Where BORG differs from the R packages that generate the same folds. [†]spatialsample, sperrorest, and mlr3spatiotempcv are fold generators with no dependency diagnosis (all “no”). “partial” marks a related diagnostic (blockCV’s autocorrelation-range estimate; CAST’s kNNDM geometry matching) offered without testing for dependence or changing the default split. Fold generation itself, which every package supports, is omitted.

Capability	BORG	blockCV	CAST	others [†]
Diagnoses the dependence	yes	partial	partial	no
Estimates CV inflation	yes	no	no	no
Blocks random CV by default	yes	no	no	no
Audits pipeline leakage	yes	no	no	no
Writes methods text	yes	no	no	no

Software design

BORG separates diagnosis from enforcement. `borg_diagnose()` tests each supplied structure: spatial autocorrelation via Moran’s I (Moran, 1950), temporal autocorrelation via the autocorrelation function and the Ljung-Box test (Ljung & Box, 1978), and grouped structure via the intraclass correlation. It returns a dependency type, a severity, an estimated inflation, and a recommended scheme. `borg_cv()` then generates the scheme; `borg()` wraps both and returns folds as a plain list or as `rsample`, `caret`, or `mlr3` objects.

The problem the diagnosis formalizes: for a model $\hat{f}$ trained on $\{(s_i, x_i, y_i)\}$ and applied at a new location s^* , the quantity CV should estimate is the generalization error

$$\text{Err} = \mathbb{E} \left[L(y^*, \hat{f}(x^*)) \right],$$

where L is the loss and s^* is drawn from the prediction domain. A CV partition into training and validation sets $\{(\mathcal{T}_k, \mathcal{V}_k)\}$ estimates Err without bias only when the validation points are independent of the training points. Under a positive dependency, $\text{Cov}(y_i, y_j) = C(\|s_i - s_j\|) > 0$ for close s_i, s_j , a random assignment scatters correlated points across folds, so validation points sit next to training points and $\widehat{\text{Err}}_{\text{CV}} < \text{Err}$. BORG tests whether $C(\cdot) > 0$ and, when it is, blocks folds so that validation points lie beyond the estimated dependence range.

A second axis guards the pipeline rather than the folds. `borg_inspect()`, `borg_validate()`, and `borg_audit()` classify evaluation risks as `hard_violation` (preprocessing fit on all data, train-test identifier overlap, outcome-derived features) or `soft_inflation` (insufficient block size, post-hoc subgroup analysis), enforcing the former and flagging the latter. `borg_aoa()` flags prediction locations outside the training feature space (Meyer & Pebesma, 2021).

Benchmark

Two experiments quantify the cost of ignoring dependence, reproducible from `paper/bench_simulation.R` and `paper/bench_spatial_cv.R`. Both use a random forest (Wright & Ziegler, 2017), single threaded, with medians over 20 seeds.

69 The primary experiment supplies the ground truth that real data cannot (Figure 1a). A
70 Gaussian random field is sampled in six spatial clusters, the common ecological sampling
71 pattern, and an independent uniform test set gives the true generalization error for mapping
72 to new locations (RMSE 0.68). Random 5-fold CV reports 0.39, understating that error by
73 42%; BORG's spatial-block CV reports 0.60, cutting the shortfall to 13%. BORG diagnosed the
74 dependency in all 20 simulations. No scheme fully recovers the extrapolation error under
75 heavy clustering, but the naive estimate is optimistic by a margin that would reverse most
76 model-selection decisions.

77 A real-data check on the Meuse dataset (Pebesma & Bivand, 2005) shows the same mechanism
78 without ground truth (Figure 1b). Predicting log zinc from distance-to-river, elevation, organic
79 matter, flood frequency, and soil class, the six 5-fold schemes report RMSE from 0.33 to
80 0.40, a 22% spread from the CV scheme alone. BORG diagnoses the dependency automatically
81 (Moran's $I = 0.124$, $p < 10^{-30}$, moderate severity) and recommends spatial blocking; CAST's
82 kNNDM returns near-random folds here, correctly, because the Meuse survey samples its
83 domain densely enough that random CV is defensible. Which end of that 22% range to trust
84 is what the diagnosis decides and the other packages leave to the user.

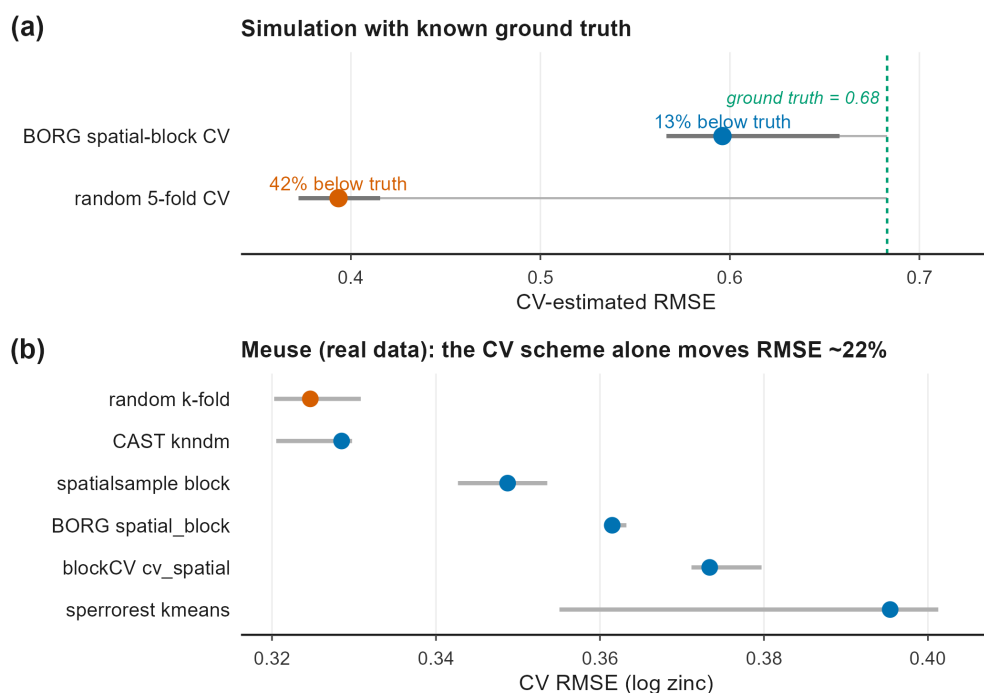


Figure 1: Cross-validation error under different schemes. (a) Simulation with known ground truth (green line): random 5-fold CV underestimates the true error by 42%, BORG spatial-block CV by 13%. (b) Six schemes on the Meuse dataset span a 22% range; orange marks random (no blocking), blue marks spatial blocking. Points are medians, bars the interquartile range over 20 seeds.

Availability

BORG is on CRAN (`install.packages("BORG")`), developed at <https://github.com/gcol33/BORG> under the MIT license, with a testthat suite, continuous integration on Linux, macOS, and Windows, and documentation at <https://gillescolling.com/BORG/>.

AI usage disclosure

The package was developed using a modern AI-assisted developer stack. The author designed the package architecture, made the algorithm-selection choices, and wrote the implementation in a TUI-first command-line workflow, using a self-hosted Qwen3-Coder-Next-REAP-48B-A3B model (mixture-of-experts coder, ~3B active per token, 4-bit MLX quantization, running on a single Apple M4 Pro) for code completion, refactoring, and boilerplate generation through Anthropic's Claude Code CLI, with requests routed to the local model and no cloud inference.

Acknowledgements

No external funding supported this work.

References

- Brenning, A. (2012). Spatial cross-validation and bootstrap for the assessment of prediction rules in remote sensing: The R package *sperrorest*. *2012 IEEE International Geoscience and Remote Sensing Symposium (IGARSS)*, 5372–5375. <https://doi.org/10.1109/IGARSS.2012.6352393>
- Kapoor, S., & Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 100804. <https://doi.org/10.1016/j.patter.2023.100804>
- Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6, 1–21. <https://doi.org/10.1145/2382577.2382579>
- Linnenbrink, J., Mila, C., Ludwig, M., & Meyer, H. (2024). kNNDM CV: K-fold nearest-neighbour distance matching cross-validation for map accuracy estimation. *Geoscientific Model Development*, 17(15), 5897–5912. <https://doi.org/10.5194/gmd-17-5897-2024>
- Ljung, G. M., & Box, G. E. P. (1978). On a measure of lack of fit in time series models. *Biometrika*, 65(2), 297–303. <https://doi.org/10.1093/biomet/65.2.297>
- Meyer, H., & Pebesma, E. (2021). Predicting into unknown space? Estimating the area of applicability of spatial prediction models. *Methods in Ecology and Evolution*, 12(9), 1620–1633. <https://doi.org/10.1111/2041-210X.13650>
- Meyer, H., Reudenbach, C., Hengl, T., Katurji, M., & Nauss, T. (2018). Improving performance of spatio-temporal machine learning models using forward feature selection and target-oriented validation. *Environmental Modelling & Software*, 101, 1–9. <https://doi.org/10.1016/j.envsoft.2017.12.001>
- Moran, P. A. P. (1950). Notes on continuous stochastic phenomena. *Biometrika*, 37(1/2), 17–23.
- Pebesma, E. J., & Bivand, R. S. (2005). Classes and methods for spatial data in R. *R News*, 5(2), 9–13.
- Roberts, D. R., Bahn, V., Ciuti, S., Boyce, M. S., Elith, J., Guillera-Aroita, G., Hauenstein, S., Lahoz-Monfort, J. J., Schröder, B., Thuiller, W., Warton, D. I., Wintle, B. A., Hartig, F., & Dormann, C. F. (2017). Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8), 913–929. <https://doi.org/10.1111/ecog.02881>
- Schratz, P., Becker, M., Lang, M., & Brenning, A. (2024). *mlr3spatiotempcv*: Spatiotemporal resampling methods for machine learning in R. *Journal of Statistical Software*, 111(7), 1–36. <https://doi.org/10.18637/jss.v111.i07>

- 132 Silge, J., & Mahoney, M. (2023). *Spatialsample: Spatial resampling infrastructure*. [https://](https://spatialsample.tidymodels.org)
133 spatialsample.tidymodels.org
- 134 Valavi, R., Elith, J., Lahoz-Monfort, J. J., & Guillerá-Arroita, G. (2019). blockCV: An
135 R package for generating spatially or environmentally separated folds for k-fold cross-
136 validation of species distribution models. *Methods in Ecology and Evolution*, 10(2),
137 225–232. <https://doi.org/10.1111/2041-210X.13107>
- 138 Wright, M. N., & Ziegler, A. (2017). Ranger: A fast implementation of random forests
139 for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1), 1–17.
140 <https://doi.org/10.18637/jss.v077.i01>

DRAFT